

Worksheet 3 — Cryptography Used Correctly (and Misused) (3 hrs)

Course: Software Security (KOSEN69) · **Week 3 Aligned to:** OWASP 2025 A04 Cryptographic Failures · CWE-327, CWE-916, CWE-330, CWE-798 **Signature game:** "Capture the Hash" (recover plaintext from weak hashes)

Ethics note: Crack only the hashes provided in `hashes.txt` on your own machine. Password-cracking against accounts or systems you don't own is illegal. Wordlists and recovered values stay inside the lab VM.

Part 1 — Student Information

Name	Student ID	Date	Group
Siravit Thakaew	6631503041		

Part 2 — Lecture Questions

Answer in your own words (2–4 sentences each).

1. Distinguish hashing, encryption, and encoding — and give one job each is the wrong tool for. Using Each for the Wrong Job

Hashing: The wrong tool for sending secret messages intended to be read by the recipient.

Reason: Hashing is a one-way function. Once data is processed, the hash value cannot be converted back into the original text; the original data is effectively lost.

Encryption: The wrong tool for storing user passwords.

Reason: Although it may appear secure, encryption is reversible (two-way). If the server or the encryption key is compromised, hackers can immediately decrypt everyone's passwords into plaintext. This task requires a one-way salted hash (e.g., bcrypt, Argon2) instead.

Encoding: The wrong tool for security or data obfuscation.

Reason: Encoding involves no data protection mechanism and uses no secret key. Anyone who recognizes the format (e.g., Base64) can immediately reverse the process; therefore, it cannot be used to prevent unauthorized access to data.

2. Why is a fast hash like MD5/SHA-1 a bad choice for storing passwords, and what should be used instead?

Brute-Force & Hardware Acceleration: Modern GPUs, ASICs, and dedicated cracking rigs can calculate billions (or trillions) of MD5/SHA-1 hashes per second. If an attacker dumps your database, they can brute-force 8-character complex passwords in a matter of minutes or hours.

Precomputed Attacks (Rainbow Tables): Fast hashes without built-in per-user salts are vulnerable to lookup tables containing billions of precomputed hashes of common passwords and wordlists.

Cryptographic Vulnerabilities: Both MD5 and SHA-1 suffer from well-documented collision vulnerabilities, rendering them completely broken for cryptographic security.

3. What is a salt, what attack does it defeat, and why must it be unique per password?

What is a Salt?

A "Salt" is a random string of data (numbers or text) generated by a secure system (CSPRNG) and appended to or combined with a user's password before it is processed by a hash function (e.g., `Hash(Password + Salt)`). This Salt is stored alongside the hash value in the database in plain view (it does not need to be kept secret).

Attacks Prevented by Salts

Rainbow Table Attacks: Hackers often pre-calculate hashes for common words and store them in large tables for comparison. When a highly random Salt is used, the resulting hash will not match any entry in these pre-computed tables, rendering Rainbow Tables ineffective.

Bulk / Batch Cracking: "Shotgun-style" database attacks aimed at cracking multiple user passwords simultaneously in a single pass become impossible. Hackers can no longer compute a single hash and compare it against the entire system's user base.

Why Must Salts Be Unique per Password?

If a system uses the same Salt globally (Global Salt) or reuses Salts:

Identical passwords yield identical hashes: If User A and User B both set their password to "Password123":

Without Salt or with reused Salt: Their hashes would look exactly the same. Hackers would immediately know they share the same password; if one person's password is cracked, the other is instantly compromised.

With Unique Salt: Even if the passwords are identical, the resulting hashes will be completely different, making it impossible to detect password similarities from the outside.

Forces hackers to compute hashes individually: Since each account has its own unique Salt... Hackers would need to execute a brute-force attack individually for each person—a one-to-one process—causing the time and cost required to compromise the entire database to multiply in direct proportion to the number of users (e.g., with one million users, the cost increases a million-fold).

4. Why does AES-ECB leak structure, and what does an authenticated mode like AES-GCM add? ECB (Electronic Codebook) is the simplest block cipher mode; it divides data into equal-sized blocks (16 bytes for AES) and encrypts each block independently using the same key.

Deterministic Encryption: If plaintext blocks are identical, the resulting ciphertext will always be the same.

Patterns are not hidden: Most real-world data contains repetitive patterns—such as identical pixel colors in an image or structural fields in files/protocols (e.g., JSON, XML)—meaning the resulting ciphertext retains the original outline or structure.

A classic example is the "ECB Penguin": when the Tux penguin image is encrypted using ECB mode, the color details change, but the penguin's outline and shape remain clearly visible.

What AES-GCM adds:

AES-GCM (Galois/Counter Mode) is classified as an AEAD (Authenticated Encryption with Associated Data) scheme. It addresses data leakage issues and adds three critical security features:

1. Randomness via Nonce/IV (Solving the Structure Leakage Problem):

GCM utilizes Counter Mode (CTR Mode) combined with an Initialization Vector (IV/Nonce) that is randomly generated for every operation.

Identical data blocks yield completely different ciphertext results each time, ensuring no original patterns or structures remain visible.

2. Data Integrity & Authenticity (Preventing Data Tampering):

Basic modes (such as ECB or CBC) only ensure confidentiality; they are vulnerable to attacks like "bit-flipping," where the secret content could be altered without the system detecting it.

GCM always generates an Authentication Tag (MAC); if even a single bit of data is altered... The decryption process immediately flags errors and rejects the data.

3. Support for Associated Data (AAD):

It can verify the integrity of header information or metadata that must remain in plaintext (such as port numbers or IP addresses) without encrypting those specific parts.

4. High Speed and Performance:

GCM computations support parallel processing on modern hardware, making it significantly faster than chaining modes like CBC.

5. What's the difference between **random** and a CSPRNG (e.g. **secrets**), and where does it matter? Key Differences

random (Standard PRNG): Designed for speed and data simulation (often using the Mersenne Twister algorithm). It is **not** cryptographically secure because it is predictable; if a hacker observes a sufficient number of outputs (e.g., 624 values in Python), they can reverse-engineer the sequence to determine all future values.

CSPRNG (e.g., **secrets**): Stands for Cryptographically Secure Pseudo-Random Number Generator. It draws entropy (random variance) directly from the operating system. It is designed to be mathematically unpredictable; even if a hacker knows all previous values, they cannot predict the next one.

Critical Risks of Choosing the Wrong One (Where it matters)

Using standard **random** for security-sensitive tasks creates critical vulnerabilities in the following scenarios:

Session IDs and CSRF Tokens: Hackers can predict token values in advance to perform session hijacking and impersonate other users.

Password Reset Tokens / Magic Links: Hackers can trigger their own password reset to calculate the sequence and then guess the reset token for an administrator account.

Cryptographic Keys, Nonces, and Salts: If used to generate IVs/Nonces for encryption (like AES-GCM) resulting in collisions, or to generate secret keys, the entire encryption system can be immediately compromised.

OTP / Verification Codes: Verification codes can be guessed without waiting for the actual transmission.

“We used AES” says nothing.

OWASP A04 · four independent decisions, four separate fixes.



One cipher name answers none of these four questions.

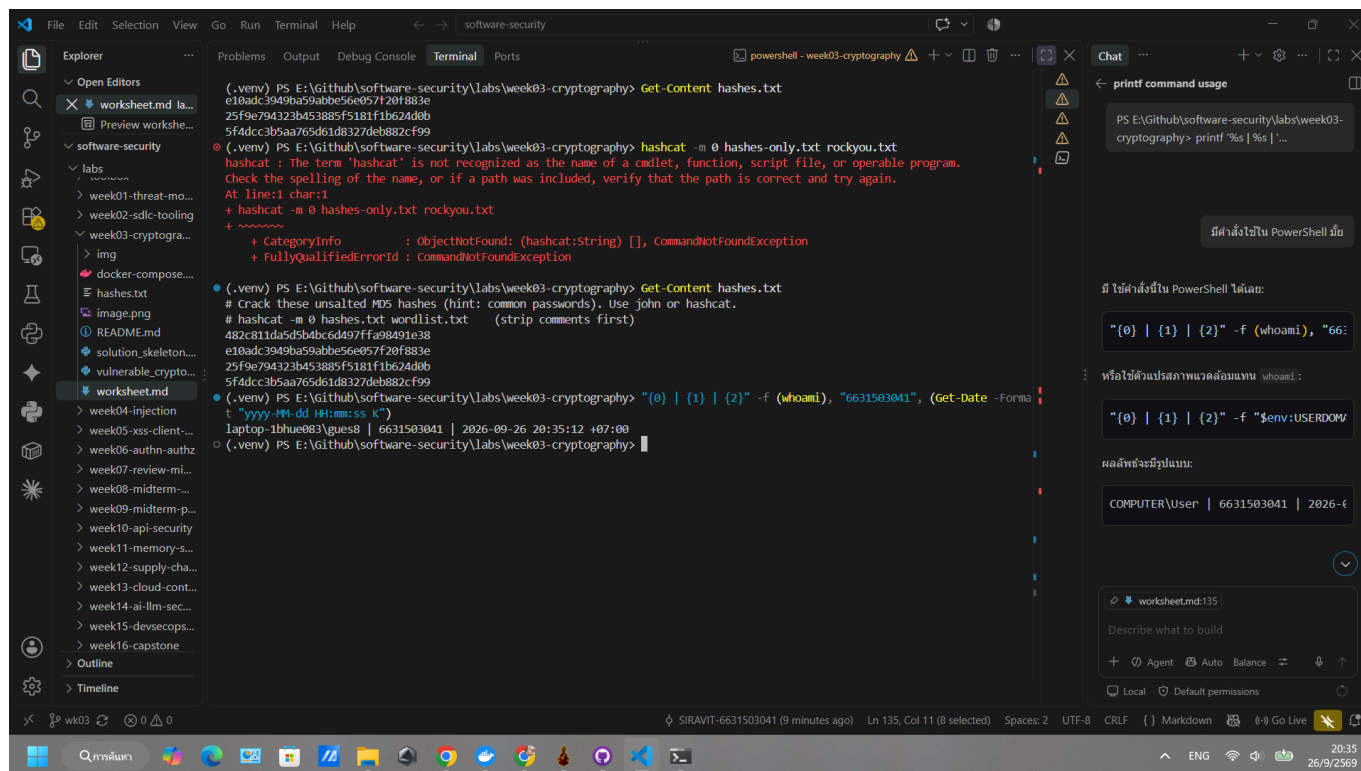
AES-GCM under a hardcoded key is still CWE-798.

Part 3 — Hands-on Lab (180 min)

Learning goals: exploit four crypto misuses, then remediate them with a vetted KDF, authenticated encryption, and a CSPRNG. **Prerequisites:** Docker (or local Python 3.12); `hashcat` or `john`; the `rockyou.txt` wordlist.

Environment setup

5 / 10



aes-modes

Task 2 — ECB structure leak (20 min) · Goal: prove ECB leaks. Steps: call `encrypt_ecb(b"A"*16 + b"A"*16)` from `vulnerable_crypto.py` and show the two 16-byte ciphertext blocks are identical; explain how this leaks plaintext structure (CWE-327). Deliverable: hex output highlighting the repeated block.

```
(.venv) PS E:\Github\software-security\labs\week03-cryptography> "{0} | {1} | {2}" -f (whoami), "6631503041", (Get-Date -Format "yyyy-MM-dd HH:mm:ss K")
laptop-1bhue83\gues8 | 6631503041 | 2026-09-26 20:35:12 +07:00 python -c "from vulnerable_crypto import encrypt_ecb; x=encrypt_ecb(b'A'*16+b'A'*16); print(x.hex()); print(x[:16].hex() == x[16:32].hex())"
3bf0d4cc0d7ed55358e2cbe19de213833bfd04cc0d7ed55358e2cbe19de21383377222e061a924c591cd9c27ea163ed4
True
(.venv) PS E:\Github\software-security\labs\week03-cryptography>
```

Identical plaintext blocks result in identical ciphertext blocks, thereby leaking data structure information.

Task 3 — Predictable token (15 min) · Goal: show the reset token is guessable. Steps: call `reset_token()` repeatedly; argue why a 6-digit random token (10^6 space, non-CSRNG) is brute-forceable (CWE-330). Deliverable: sample tokens + a one-line attack estimate.

```
(.venv) PS E:\Github\software-security\labs\week03-cryptography> python -c "from vulnerable_crypto import reset_token; print([reset_token() for _ in range(10)])"
['469222', '695941', '406306', '696043', '019792', '353711', '471676', '256144', '472714', '876204']
(.venv) PS E:\Github\software-security\labs\week03-cryptography> "{0} | {1} | {2}" -f (whoami), "6631503041", (Get-Date -Format "yyyy-MM-dd HH:mm:ss K")
laptop-1bhue83\gues8 | 6631503041 | 2026-09-26 20:38:42 +07:00
```

The token has only 10^6 possible values and uses a PRNG unsuitable for security purposes, making it susceptible to guessing or brute-force attacks.

Task 4 — Hardcoded key (5 min) · Goal: identify the key-management flaw. Steps: find `HARDCODED_KEY` in `vulnerable_crypto.py`; explain why shipping a key in source is CWE-798. Deliverable: the line + a 2-sentence mitigation. The problem is that anyone with access to the source code immediately obtains the encryption key. The solution is to load the key from an environment variable or a secret manager.

Task 5 — Crack the project target's hashes (25 min) · Goal: apply cracking to your term project. Steps:

NoteVault stores unsalted MD5 password hashes; obtain them (via the app's `/admin` once you can reach it, or from its `seed()`), and crack them with `hashcat -m 0`. Deliverable: the recovered password(s) + note the CWE — record this finding for your project report (`project/REPORT-TEMPLATE.md` in the repo root).

```
(.venv) PS E:\Github\software-security\labs\week03-cryptography> "{0} | {1} | {2}" -f (whoami), "6631503041", (Get-Date -Format "yyyy-MM-dd HH:mm:ss K")
laptop-1bhue083\gues8 | 6631503041 | 2026-09-26 20:38:42 +07:00
(.venv) PS E:\Github\software-security\labs\week03-cryptography> Select-String -Path ..\..\project\starter-app\app.py -Pattern "md5"

E:\Github\software-security\project\starter-app\app.py:68:      [("alice",
hashlib.md5(b"alicepw").hexdigest(), "user"),
E:\Github\software-security\project\starter-app\app.py:69:      ("admin",
hashlib.md5(b"admin123").hexdigest(), "admin"))
E:\Github\software-security\project\starter-app\app.py:117:      (username,
hashlib.md5(password.encode()).hexdigest(), role))
E:\Github\software-security\project\starter-app\app.py:129:      username, hashlib.md5((password or
 "").encode()).hexdigest())
```

CWE-327/CWE-916 due to the use of unsalted MD5.

Task 6 — Password storage migration (25 min) · Goal: fix it the way real apps do. Steps: write

`store_password/verify_password` with `argon2id`, and a **rehash-on-login** path that upgrades a legacy MD5 record to argon2id the next time the user logs in. Deliverable: the code + a short note on why migration matters.

```
from argon2 import PasswordHasher
```

```
ph = PasswordHasher()
```

```
def store_password(pw: str) -> str: return ph.hash(pw)
```

```
def verify_password(hash_: str, pw: str) -> bool: try: return ph.verify(hash_, pw) except Exception: return False
```

Argon2id automatically generates a salt.

Task 7 — Authenticated encryption round-trip (20 min) · Goal: use AEAD correctly. Steps: encrypt+decrypt a message with **AES-GCM** using a random 12-byte nonce and a key from an env var; then flip one ciphertext byte and show decryption **fails** (tag check). Deliverable: the round-trip output + the tampered-fails proof.

```
nonce, ciphertext, tag = encrypt_gcm(b"secret", key) print(decrypt_gcm(nonce, ciphertext, tag, key))
```

```
tampered = bytearray(ciphertext) tampered[0] ^= 1
```

```
try: decrypt_gcm(nonce, bytes(tampered), tag, key) print("ERROR: tampering accepted") except ValueError:
print("tampering rejected")
```

```
b'secret' tampering rejected
```

Task 8 — TLS in practice (15 min) · Goal: read a real cert. Steps: run `openssl s_client -connect`

`example.com:443 </dev/null 2>/dev/null | tee /tmp/tls.txt | openssl x509 -noout -issuer -subject -dates` for the cert summary, then `grep -E 'Protocol|New,' /tmp/tls.txt` for the negotiated TLS version (the version line is printed by `s_client`, not by `x509`, so the plain pipe would discard it); identify issuer, validity, and that TLS version. Deliverable: the cert summary + one line on what TLS protects that hashing/at-rest encryption does not.

Notes: Issuer, Subject, Start and expiration dates, TLS version, ,TLS protects data in transit, whereas hashing is used for verification/integrity checks, and encryption-at-rest protects stored data.

Task 9 — Defend / fix it (20 min) · *Goal:* remediate using `solution_skeleton.py`. *Steps:* run `python solution_skeleton.py`; confirm `store_password/verify_password` use argon2id (auto-salted), `encrypt_gcm` uses a random 12-byte nonce + auth tag with a key from `ENC_KEY_HEX` env, and `reset_token` uses `secrets`. Map each fix to the CWE it closes. *Deliverable:* before/after table (misuse → fix → CWE closed) + screenshot of the fixed script running.

```
PS E:\Github\software-security\labs\week03-cryptography> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ;
(& e:\Github\software-security\.venv\Scripts\Activate.ps1)
(.venv) PS E:\Github\software-security\labs\week03-cryptography> $env:ENC_KEY_HEX="00112233445566778899aabbccddeeff00112233445
566778899aabbccddeeff"
(.venv) PS E:\Github\software-security\labs\week03-cryptography> python solution_skeleton.py
argon2 ok: True
gcm: (b'\x15\xb3`l\xa8d\xa3c\xce\x7fs=}', b"s+p'Y\xfs", b'F,\xfd\x9fGA\xfc\xd6E\xec\xcc0\x03h\x80e')
token: 5FHotFB9J438lidJfDyBdw
(.venv) PS E:\Github\software-security\labs\week03-cryptography> "{0} | {1} | {2}" -f (whoami), "6631503041", (Get-Date -Forma
t "yyyy-MM-dd HH:mm:ss K")
laptop-1bhue083\gues8 | 6631503041 | 2026-09-26 20:51:55 +07:00
(.venv) PS E:\Github\software-security\labs\week03-cryptography>
```

Part 4 — Reflection

1. Map each of the four misuses to its CWE and to OWASP A04, in one line each.

Weak Password Hashing (e.g., MD5/SHA-1): Maps to CWE-916 (Insufficient Computational Effort) and OWASP A04:2025 (Cryptographic Failures) because fast, unsalted algorithms allow rapid offline cracking.

Hardcoded Cryptographic Keys: Maps to CWE-798 (Hard-coded Credentials) and OWASP A04:2025 because embedding secrets in source code completely compromises the encryption's root of trust.

Insecure Block Cipher Mode (ECB): Maps to CWE-327 (Broken/Risky Crypto Algorithm) and OWASP A04:2025 because deterministic encryption fails to hide data patterns, breaking confidentiality.

Predictable Randomness (Weak PRNG/IVs): Maps to CWE-330 (Insufficiently Random Values) and OWASP A04:2025 because standard math functions allow attackers to predict keys or initialization vectors.

2. Name a real-world breach caused by weak password hashing or hardcoded keys, and which fix here would have prevented it.

Breach: LinkedIn (2012)

Root Cause: Weak password hashing. LinkedIn stored user passwords as unsalted SHA-1 hashes, which are incredibly fast to compute.

The Fix: Applying the bcrypt/Argon2 hashing fix (which includes a unique salt and a slow computational work factor) would have rendered the stolen database largely useless, protecting the passwords from the rapid rainbow table and brute-force attacks that cracked millions of accounts within days.

3. Across all four fixes, which closes the largest real-world risk, and why?

Removing the hardcoded cryptographic key closes the largest real-world risk.

While weak algorithms (like ECB mode or SHA-1) require an attacker to capture ciphertext and expend at least some computational effort to crack hashes or analyze patterns, a hardcoded key represents a complete bypass of the cryptographic system. If an attacker views a leaked commit, reverse-engineers a mobile binary, or accesses the repository, they instantly obtain the exact key needed to decrypt all data in plaintext—requiring zero cryptographic skill or cracking time.

Grading rubric (100)

Criterion	Points
Lecture questions (Part 2)	20
Exploitation + evidence (cracked hashes + ECB/token/key proof + screenshots)	40
Defense (working <code>solution_skeleton.py</code> + before/after mapping)	25
Reflection (CWE/OWASP mapping + breach + biggest-risk fix)	15

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence.** When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. This blank is your personal record only; the flag itself is scored by submitting it in the `ctf.zcr.ai` challenge — the worksheet PDF is a separate submission, to `learn.zcr.ai/submit` (full guide: `SUBMISSION.md` in the repo root).*
- **Explain in your own words** (graded on your reasoning, not copied text):
 1. What did you do, and **why did the vulnerability work?**
 2. **Why does your fix actually stop it** — and what could still break it?



Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.



Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*